

Объектно-ориентированное программирование в Python

В программировании большинство языков предлагают различные функции, которые дают нам разные способы решения технических задач. Поскольку существует так много разных языков со своим уникальным набором функций, возникла необходимость создать систему классификации, помогающую различать эти наборы функций. В конечном итоге это привело к созданию термина парадигма программирования — способа классификации различных языков программирования и уникальных функций, которые они предлагают.

По мере того, как мы глубже изучаем Python, наш код может одновременно относиться к нескольким категориям парадигм. Это связано с тем, что большинство современных языков предлагают более одной конкретной парадигмы, в которой мы можем программировать. Хотя мы могли бы целый день изучать все парадигмы, которые предлагает Python, вместо этого мы погрузимся в одну из самых популярных парадигм, которую мы, возможно, уже использовали (возможно, неосознанно), под названием Объектно-ориентированное программирование (ООП).

В основе любого языка, классифицируемого как ООП-язык, должна существовать возможность создавать программы вокруг классов и объектов. Мы уже начали работать с этими концепциями ранее, когда создавали собственные пользовательские классы, методы класса и объекты экземпляров. Для повторения давайте посмотрим на пример:

```
>>>class Dog:
>>>    sound = "Woof"
>>>
>>>    def __init__(self, name, age):
>>>        self.name = name
>>>        self.age = age
>>>
>>>    def bark(self):
>>>        print(Dog.sound)
```

В приведенном выше примере мы представляем реальную сущность (собаку) как класс со свойствами (name и age) и методами (bark). Эти функции составляют ядро парадигмы ООП и в конечном итоге позволяют нам создавать более сложные программы. Однако это лишь поверхностно затрагивает то, чего мы можем достичь. Чтобы глубже изучить парадигму, мы рассмотрим четыре основных столпа ООП:

- Наследование (Inheritance)
- Полиморфизм (Polymorphism)
- Абстракция (Abstraction)
- Инкапсуляция (Encapsulation)

Каждый из этих столпов позволит нам расширить наши навыки, чтобы мы могли в полной мере использовать мощь объектно-ориентированного программирования в Python! Мы начнем изучать эти столпы в последующих упражнениях, а пока давайте освежим основы ООП.

Задание

1. Чтобы начать наше исследование ООП, создайте класс, который будет представлять сотрудника компании.
 1. Определите класс Employee с методом `__init__()`.
 2. Определите переменную класса `new_id` и установите её равной 1.
2. Каждому экземпляру Employee потребуется свой уникальный ID. Внутри класса Employee:
 1. Определите метод `__init__()`.
 2. Внутри `__init__()` определите `self.id` и установите его равным переменной класса `new_id`.
 3. Наконец, увеличьте `new_id` на 1.
3. Теперь создайте функцию для вывода ID экземпляра. Внутри класса Employee:
 1. Определите метод `say_id()`.
 2. Внутри `say_id()` выведите строку "My id is " и затем ID экземпляра.
4. Наконец, создайте 2 сотрудников и заставьте их сообщить свои ID. Вне класса Employee:
 1. Определите переменную `e1` и установите её равной экземпляру Employee.
 2. Определите переменную `e2` и установите её равной экземпляру Employee.
 3. Заставьте оба `e1` и `e2` вывести свои ID.

Столп ООП: Наследование

Когда мы слышим слово «наследование», код может быть не первым, что приходит на ум; мы, вероятно, скорее подумаем о наследовании генетических черт, цвета глаз от матери или ямочек от дедушки. В мире объектно-ориентированного программирования наследование на самом деле является одним из основных столпов для создания сложных структур с нашими классами. Чтобы погрузиться в эту концепцию, давайте рассмотрим классы Dog и Cat:

```
>>class Dog:
    >>def bark(self):
        >> print('Woof!')

>>class Cat:
    >>def meow(self):
        >>print('Meow!')
```

Эти два класса определяют двух различных животных со своими собственными методами общения. Теперь, что если мы хотим дать обоим этим классам возможность есть, вызывая метод под названием `eat()`. Мы могли бы написать метод дважды в обоих классах, но тогда мы бы повторяли код! Нам также, возможно, придется писать его внутри каждого конкретного класса животных, который мы когда-либо создадим. Вместо этого мы можем использовать силу наследования.

Поскольку и Cat, и Dog подпадают под классификацию Animal, мы можем создать **родительский класс**, чтобы представить свойства и методы, которыми они оба могут поделиться! Вот как это может выглядеть:

```
>>class Animal:
>>def eat(self):
>>print("Nom Nom Nom...eating food!")
```

Отлично, у нас есть класс `Animal` с методом `eat()`, но как нам фактически заставить классы `Dog` и `Cat` **унаследовать** этот метод, чтобы он мог быть общим для обоих классов? Ну вот, базовая структура будет выглядеть так:

```
>>class ParentClass:
>># методы/свойства класса...
>>class ChildClass(ParentClass):
>> # методы/свойства класса...
```

Если мы применим эту структуру к нашему примеру, наш код будет выглядеть так:

```
>>class Dog(Animal):
>>def bark(self):
>>print('Bark!')

>>class Cat(Animal):
>>def meow(self):
>>print('Meow!')
```

Теперь давайте посмотрим на наследование в действии:

```
>>fluffy = Dog()
>>zoomie = Cat()

>>fluffy.eat() # Nom Nom Nom...eating food!
>>zoomie.eat() # Nom Nom Nom...eating food!
```

Как мы видим, есть некоторые явные преимущества использования наследования. Мало того, что мы можем повторно использовать методы в нескольких классах, используя наш **родительский класс**, но мы также можем создавать отношения родитель-потомок между сущностями!

Задание

1. Теперь, когда есть класс `Employee`, мы хотим сделать более конкретный тип сотрудника, `Admin`.
 1. Создайте класс `Admin`, который наследуется от класса `Employee`.
 2. Внутри тела класса вставьте оператор `pass`.
2. Теперь пришло время протестировать вашу реализацию наследования. Внизу
 1. Определите переменную `e3` и установите её равной экземпляру класса `Admin`. Теперь, если вы вызовете метод `.say_id()` экземпляра `Admin` в `e3`, вы получите вывод с ID экземпляра.

Переопределение методов

При реализации наследования дочерний класс может захотеть изменить поведение метода своего родительского класса. В Python все, что нам нужно сделать, это **переопределить** определение метода. Переопределенный метод в подклассе — это метод, который имеет то же определение, что и родительский класс, но содержит другое поведение.

```
>>>class Animal:
>>>def __init__(self, name):
>>>self.name = name

>>>def make_noise(self):
>>>print("{} says, Grrrr".format(self.name))

>>>pet1 = Animal("Rex")
>>>pet1.make_noise() # Rex says, Grrrr
```

Класс `animal` выше имеет один атрибут, `self.name`, и один метод, `.make_noise()`. Метод `.make_noise()` выводит несколько общий звук животного, "Rex says, Grrrr". Если мы определим подкласс `Animal`, мы можем захотеть издать другой звук.

```
>>>class Cat(Animal):
>>>def make_noise(self):
>>>print("{} says, Meow!".format(self.name))

>>>pet2 = Cat("Maisy")
>>>pet2.make_noise() # Maisy says, Meow!
```

Теперь мы создали класс для более конкретного типа животного, `Cat`. Он имеет все атрибуты и методы `Animal`. Однако, если вы вызовете метод `.make_noise()` на этом экземпляре `Cat`, он скажет "Maisy says, Meow!".

Задание

Как администратор, вы чувствуете, что не важно сообщать свой ID, а просто дать другим знать, что они говорят с администратором. Внутри класса `Admin`:

1. Определите метод `say_id()`.
2. Внутри метода выведите "I am an Admin". Теперь, когда вы вызовете `.say_id()` с `e3`, вы должны увидеть вывод метода `.say_id()` от `Admin`.

super()

При переопределении **методов** мы иногда хотим все еще иметь доступ к поведению родительского метода. Для этого нам нужен способ вызвать метод родительского класса. Python дает нам способ сделать это, используя `super()`.

`super()` дает нам **прокси-объект**. С этим прокси-объектом мы можем вызвать метод родительского класса объекта (также называемого его суперклассом). Мы вызываем требуемую функцию как метод на `super()`:

```
>>class Animal:
    >>def __init__(self, name, sound="Grrrr"):
        >>self.name = name
        >>self.sound = sound

    >>def make_noise(self):
        >>print("{} says, {}".format(self.name, self.sound))

>>class Cat(Animal):
    >>def __init__(self, name):
        >>super().__init__(name, "Meow!")

>>pet_cat = Cat("Rachel")
>>pet_cat.make_noise() # Rachel says, Meow!
```

В приведенном выше примере у нас есть класс `Animal` и подкласс `Cat`. `Animal` имеет 2 атрибута, `name` и `sound`, и один метод, `.make_noise()`. Метод `.make_noise()` выводит `name` и `sound` экземпляра.

Подкласс `Cat` имеет метод `.__init__()`, что означает, что метод `.__init__()` его суперкласса, `Animal`, не будет вызван при создании экземпляра `Cat`. Метод `.__init__()` из подкласса переопределяет метод из суперкласса.

Чтобы все еще вызвать метод `.__init__()` от `Animal`, `super().__init__(name, "Meow!")` вызывается внутри метода `.__init__()` подкласса. Эта дополнительная логика позволяет нам добавить звук "Meow" изнутри класса `Cat`, но все же использовать метод `.__init__()` класса `Animal`.

`super()` используется в подклассах для вызова необходимого поведения от суперкласса наряду с поведением метода подкласса.

Задание

Как только менеджеры узнали, что администраторы ходят вокруг и просто говорят людям, что они администраторы, менеджеры вмешались и заставили их также называть свой ID. Внутри класса `Admin`:

1. Добавьте строку, которая также вызывает метод `.say_id()` класса `Employee`. Теперь вывод должен содержать ID администратора и то, что он является администратором.

Давайте теперь посмотрим на возможность, реализованную в Python, под названием **множественное наследование**. Как вы, возможно, догадались из названия, это когда подкласс наследуется более чем от одного суперкласса. Одна форма множественного наследования — это когда есть несколько уровней наследования. Это означает, что класс наследует члены от своего суперкласса и своего супер-суперкласса.

```
>>class Animal:
    >>def __init__(self, name):
        >>self.name = name

    >>def say_hi(self):
        >> print("{} says, Hi!".format(self.name))

>>class Cat(Animal):
    >>pass

>>class Angry_Cat(Cat):
    >>pass

>>my_pet = Angry_Cat("Mr. Cranky")
>>my_pet.say_hi() # Mr. Cranky says, Hi!
```

В приведенном выше примере `Angry_Cat` наследуется от `Cat`, а `Cat` наследуется от `Animal`. И `Angry_Cat`, и `Cat` имеют доступ к атрибуту `name` класса `Animal` и методу `.say_hi()`. К любой функции, добавленной в `Cat`, `Angry_Cat` также будет иметь доступ.

Задание

Менеджеры решают начать ходить вокруг больше, чтобы люди знали, кто главный. Определите класс `Manager` и заставьте его наследоваться от класса `Admin`.

1. Внутри класса `Manager` определите метод `say_id()`, который выводит, что они отвечают за дело.

Менеджеры хотят подать хороший пример, поэтому они также дают людям знать свой ID и то, что они являются администраторами. Внутри метода `.say_id()` класса `Manager`:

1. Вызовите метод `.say_id()` класса `Admin`.
Теперь протестируйте это.
1. Определите переменную `e4` и установите её равной экземпляру класса `Manager`.
2. Вызовите метод `.say_id()` экземпляра в `e4`. Теперь вы получите вывод от всех 3 классов.

Множественное наследование: Часть 2

Другая форма множественного **наследования** включает подкласс, который наследуется непосредственно от двух классов и может использовать атрибуты и методы обоих.

```
>>class Animal:
    >>def __init__(self, name):
        >>self.name = name

>>class Dog(Animal):
    >>def action(self):
        >>print("{} wags tail. Awwww".format(self.name))

>>class Wolf(Animal):
    >>def action(self):
        >>print("{} bites. OUCH!".format(self.name))

>>class Hybrid(Dog, Wolf):
    >>def action(self):
        >>super().action()
        >>Wolf.action(self)

>>my_pet = Hybrid("Fluffy")
>>my_pet.action() # Fluffy wags tail. Awwww
                  # Fluffy bites. OUCH!
```

Приведенный выше пример показывает, что класс Hybrid является подклассом как Dog, так и Wolf, которые также являются подклассами Animal. Все 3 подкласса могут использовать функции в Animal, и Hybrid может использовать функции Dog и Wolf. Но Dog и Wolf **не могут** использовать функции друг друга.

Эта форма множественного наследования может быть полезной, добавляя функциональность из класса, который не вписывается в текущую схему дизайна текущих классов.

Следует проявлять осторожность при создании структуры наследования *seperiti ini*, особенно при использовании метода `super()`. В приведенном выше примере вызов `super().action()` внутри класса Hybrid вызывает метод `.action()` класса Dog. Это связано с тем, что он *listed before Wolf* в определении `Hybrid(Dog, Wolf)`.

Строка `Wolf.action(self)` вызывает метод `.action()` класса Wolf. Важная вещь, которую следует отметить здесь, заключается в том, что `self` передается как аргумент. Это гарантирует, что метод `.action()` в Wolf получает экземпляр класса Hybrid для вывода правильного `name`.

Задание

Администраторам в компании нужен доступ к потребительскому веб-сайту. Это означает, что администраторы также должны быть пользователями сайта. Класс User был

добавлен и имеет атрибуты `username` и `role` и метод `.say_user_info()`. Чтобы получить администраторам доступ пользователя, который им нужен:

1. Заставьте класс `Admin` наследоваться от класса `User` наряду с классом `Employee`. Убедитесь, что класс `Employee` listed first в определении класса `Admin`.

Теперь давайте убедимся, что администраторы получают свои пользовательские данные настроенными. Внутри метода `.__init__()` класса `Admin`:

1. Вызовите метод `.__init__()` класса `User`.
2. Передайте экземпляр класса `Admin`, `id` и строку `"Admin"` в качестве аргументов вызову метода `.__init__()`.

Подтвердите, что пользовательские данные настроены правильно.

1. Вызовите метод `.say_user_info()`, используя экземпляр `Admin` в `e3`.

Столп ООП: Полиморфизм

В компьютерном программировании **полиморфизм** — это способность применять идентичную операцию к разным типам объектов. Это может быть полезно, когда тип объекта может быть неизвестен во время выполнения программы. Полиморфизм может быть применен с использованием Python несколькими способами. Мы уже испытали его форму при изучении наследования.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def make_noise(self):
        print("{} says, Grrrr".format(self.name))

class Cat(Animal):
    def make_noise(self):
        print("{} says, Meow!".format(self.name))

class Robot:
    def make_noise(self):
        print("beep.boop...BEEEEEP!!!")

an_animal = Animal("Bear")
my_pet = Cat("Maisy")
my_vacuum = Robot()
objects = [an_animal, my_pet, my_vacuum]

for o in objects:
    o.make_noise()

# ВЫВОД
# "Bear says, Grrrr"
# "Maisy says, Meow!"
# "beep.boop...BEEEEEP!!!"
```


С экземплярами классов, добавленными в список, мы можем Iterate через список и вызывать `.make_noise()`. Это делается без необходимости знать, какому типу класса принадлежит `.make_noise()`.

Задание

Встреча должна быть запланирована по крайней мере с одним сотрудником, одним администратором и одним менеджером.

1. Определите переменную `meeting` и установите её равной списку, который содержит экземпляр каждого класса, `Employee()`, `Admin()` и `Manager()`.

С разными типами сотрудников на встрече, заставьте их всех сказать свой ID.

1. Используя цикл `for`, Iterate через список `meeting`.
2. Используя вашу определенную переменную цикла, вызовите метод `.say_id()` на каждом экземпляре в списке.

Dunder методы (Магические методы)

Код ниже показывает, что при работе с разными типами объектов, like `int`, `str` или `list`, оператор `+` выполняет разные функции. Это известно как **перегрузка операторов** и является другой формой полиморфизма.

```
>># Для int и int, + возвращает int
>>2 + 4 == 6
```

```
>># Для string и string, + возвращает string
>>"Is this " + "addition?" == "Is this addition?"
```

```
>># Для list и list, + возвращает list
>>[1, 2] + [3, 4] == [1, 2, 3, 4]
```

Чтобы реализовать это поведение, мы должны сначала обсудить **dunder методы**. Каждый определенный класс в Python имеет доступ к группе этих специальных методов. Мы уже изучили несколько из них, конструктор `__init__()` и метод строкового представления `__repr__()`. Название dunder метод происходит от **Double UNDERscores** (двойных подчеркиваний), которые окружают имя каждого метода.

Вспомните, что метод `__repr__()` принимает только один параметр, `self`, и должен возвращать строковое значение. Возвращаемое значение должно быть **строковым представлением** класса, которое можно увидеть, используя `print()` на экземпляре класса. Просмотрите пример кода ниже для примера того, как используется этот метод.

Определение dunder методов класса — это способ выполнить перегрузку операторов.

```
>>class Animal:
>>def __init__(self, name):
```

```

>>self.name = name

>> def __repr__(self):
>> return self.name

>> def __add__(self, another_animal):
>> return Animal(self.name + another_animal.name)

>>a1 = Animal("Horse")
>>a2 = Animal("Penguin")
>>a3 = a1 + a2
>>print(a1) # Печатает "Horse"
>>print(a2) # Печатает "Penguin"
>>print(a3) # Печатает "HorsePenguin"

```

Приведенный выше код имеет класс `Animal` с dunder методом, `__add__()`. Это определяет поведение оператора `+` при использовании на объектах этого типа класса. Метод возвращает новый объект `Animal` с именами объектов операндов, сцепленными вместе. В этом примере мы создали "HorsePenguin"!

Строка кода `a3 = a1 + a2` вызывает метод `__add__()` левого операнда, `a1`, с правым операндом `a2`, переданным в качестве аргумента. Атрибуты `name` из `a1` и `a2` сцепляются с использованием параметров `__add__()`, `self` и `another_animal`. Полученная строка используется как имя нового объекта `Animal`, который возвращается, чтобы стать значением `a3`.

Задание

Теперь есть класс `Meeting` с атрибутом списка `attendees` и dunder методом `__add__()`, который добавляет экземпляры `Employee` в список `attendees`. Прежде чем мы попробуем добавить сотрудников на встречу, мы хотим убедиться, что мы можем знать, сколько сотрудников находится на встрече. Внутри класса `Meeting`:

1. Перегрузите операцию `len()`, определив dunder метод `__len__()`.
2. Внутри определения `__len__()` верните длину атрибута `attendees`.

Теперь добавьте трех сотрудников на встречу:

1. Используя экземпляр `Meeting m1`, добавьте каждый из экземпляров сотрудников `e1`, `e2` и `e3`. Используйте **одну строку** для каждого экземпляра сотрудника.
2. Выведите длину экземпляра встречи `m1`. Вы должны увидеть вывод от каждого добавленного сотрудника, а затем длину встречи, 3.

Столп ООП: Абстракция

Когда программа начинает становиться большой, классы могут начать разделять функциональность, или мы можем упустить из виду цель структуры наследования класса. Чтобы облегчить такие проблемы, мы можем использовать концепцию **абстракции**.

Абстракция помогает с дизайном кода, определяя необходимые поведения, которые должны быть реализованы внутри структуры класса. Поступая так, абстракция также

помогает избежать пропуска или перекрытия функциональности класса, поскольку иерархии классов становятся больше.

```
>>from abc import ABC, abstractmethod

>>class Animal(ABC):
>>    def __init__(self, name):
>>        self.name = name

>>@abstractmethod
>>def make_noise(self):
>>    pass

>>class Cat(Animal):
>>    def make_noise(self):
>>        print("{} says, Meow!".format(self.name))

>>class Dog(Animal):
>>    def make_noise(self):
>>        print("{} says, Woof!".format(self.name))

>>kitty = Cat("Maisy")
>>doggy = Dog("Amber")
>>kitty.make_noise() # "Maisy says, Meow!"
>>doggy.make_noise() # "Amber says, Woof!"
```

Выше у нас есть классы Cat и Dog, которые наследуются от Animal. Класс Animal теперь наследуется от импортированного класса ABC, что означает Abstract Base Class (Абстрактный базовый класс).

Это первый шаг к тому, чтобы сделать Animal абстрактным классом, который не может быть instantiated. Второй шаг — использование импортированного декоратора @abstractmethod на пустом методе .make_noise().

Приведенная ниже строка кода выдаст ошибку.

```
>>an_animal = Animal("Scruffy")
>># TypeError: Can't instantiate abstract class Animal with abstract method make_noise
```

Процесс абстракции определяет, что такое Animal, но не позволяет создать его. Метод .__init__() все еще требует имени, поскольку мы чувствуем, что все животные заслуживают имени.

Метод .make_noise() существует, поскольку все животные издают какую-то форму шума, но метод не реализован, поскольку каждое животное издает разный шум. Каждый подкласс Animal теперь требуется определить свой собственный метод .make_noise(), иначе произойдет ошибка.

Это некоторые из способов, которыми абстракция поддерживает дизайн организованной структуры класса.

Задание

Определите абстрактный класс `AbstractEmployee`. Пусть он имеет всю логику, которая ранее существовала в классе `Employee`, за исключением того, что метод `.say_id()` не реализован и имеет декоратор `@abstractmethod`. Класс `Employee` в настоящее время становится пустой и не имеет реализации. Запустите код и наблюдайте, что `e1.say_id()` вызывает `AttributeError`, поскольку класс `Employee` не имеет реализации.

Давайте исправим эту ошибку:

1. Заставьте класс `Employee` наследоваться от класса `AbstractEmployee`.

Отличная работа! Но подождите, все еще есть ошибка! `TypeError: Can't instantiate abstract class Employee with abstract methods say_id` Метод `.say_id()` в классе `AbstractEmployee` использует декоратор `@abstractmethod`. Это означает, что любой класс, который наследуется от `AbstractEmployee`, должен реализовать метод `.say_id()`. Внутри класса `Employee` замените оператор `pass`, затем:

1. Определите метод `say_id()`, который выводит сообщение с `self.id`. Когда закончите, вы должны увидеть вывод в консоли.

Столп ООП: Инкапсуляция

Инкапсуляция — это процесс сокрытия методов и данных внутри объекта, к которому они относятся. Языки реализуют это с помощью так называемых **модификаторов доступа**, таких как:

1. `Public` (Публичный)
2. `Protected` (Защищенный)
3. `Private` (Приватный)

В общем, публичные члены могут быть доступны отовсюду, защищенные члены могут быть доступны только из кода внутри того же модуля, а приватные члены могут быть доступны только из кода внутри класса, где эти члены определены.

В Python нет встроенного механизма для предотвращения доступа к любому члену (т. е. все члены в Python являются публичными). Однако среди разработчиков существует общее соглашение использовать одинарное подчеркивание `self._x`, чтобы указать, что член является защищенным. Доступ к защищенному члену вне модуля не вызовет ошибки, это добавлено разработчиками, чтобы информировать других разработчиков, что они должны быть осторожны при доступе к этому члену таким образом.

Аналогично, мы можем объявить член как приватный с двумя ведущими подчеркиваниями `self.__x`. Это больше, чем просто соглашение в Python, из-за механизма, называемого **искажением имен (name mangling)**. Члены, перед которыми стоят два подчеркивания, изменяют свои имена в фоновом режиме на `obj._Classname__x`. Хотя к ним все еще можно получить доступ публично, цель этого механизма — предотвратить конфликт имен членов любых наследующих классов, которые могут определить член с тем же именем.

Обратите внимание, что это отличается от dunder-методов, которые мы обсуждали ранее. Dunder-метод имеет два ведущих и два завершающих подчеркивания и обрабатывается иначе, чем приватный член. Одно важное отличие заключается в том, что имена dunder-методов не искажаются.

Задание

Класс `Employee` содержит один атрибут `id`. Переменная экземпляра `e` определена и затем передана функции `dir()`, которая выводится в консоль. `dir()` — это встроенная функция Python, которая возвращает список всех членов класса, включая dunder-методы. Когда вы запустите код, вы увидите список членов класса с `id` в качестве последнего элемента.

Теперь добавьте атрибут, который использует соглашение именования с одинарным подчеркиванием. Внутри метода `__init__()` класса `Employee`:

1. Определите переменную с одинарным подчеркиванием `_id` и установите её равной чему угодно. Когда вы запустите код, вы можете увидеть `_id` в качестве предпоследнего элемента в списке вывода.

Теперь определите переменную, используя двойное подчеркивание. Внутри метода `__init__()` класса `Employee`:

1. Определите переменную с двойным подчеркиванием `__id` и установите её равной чему угодно. Когда вы запустите код, вы можете увидеть новую переменную `__Employee__id` в качестве первого элемента в списке вывода. Это результат искажения имен (name-mangling) переменной `self.__id`.

Геттеры, Сеттеры и Делитеры

Использование функций **getter**, **setter** и **deleter** — это один из способов реализации инкапсуляции внутри Python, где состояние атрибутов класса может быть обработано внутри класса. Эти функции полезны для обеспечения того, чтобы обрабатываемые данные были подходящими для определенной функциональности класса.

```
class Animal:
    def __init__(self, name):
        self._name = name
        self._age = None

    def get_age(self):
        return self._age

    def set_age(self, new_age):
        if isinstance(new_age, int):
            self._age = new_age
        else:
            raise TypeError

    def delete_age(self):
        print("__age Deleted")
        del self._age
```

Глядя на класс `Animal` выше, есть атрибут `_age` с одинарным подчеркиванием. Это обозначает, что он предназначен для использования только внутри модуля. Затем есть 3

метода, related to age, каждый с разной целью. Они определяют getter, setter и deleter конкретного свойства.

Первый метод, related to age, является getter и возвращает self._age. Setter реализован ниже. Он включает логику, которая гарантирует, что значение, переданное в new_age, является integer. Если так, self._age = new_age. Если нет, raise an error. Это полезно и показывает силу использования этих функций для инкапсуляции.

Deleter реализован ниже setter. Он выводит сообщение подтверждения и использует ключевое слово del для удаления атрибута self._age.

```
a = Animal("Rufus")
print(a.get_age()) # None

a.set_age(10)
print(a.get_age()) # 10

a.set_age("Ten") # Raises a TypeError

a.delete_age() # "_age Deleted"
print(a.get_age()) # Raises a AttributeError
```

Выше мы видим, что a.get_age() получает значение _age, a.set_age(10) устанавливает значение, и a.delete_age() удаляет атрибут entirely. TypeError происходит с a.set_age("Ten"), потому что определенная логика в setter looking only for an integer. AttributeError происходит с a.get_age() после того, как атрибут был удален.

Задание

Используя getter, setter и соглашение именования с одинарным подчеркиванием, сотрудники компании теперь смогут использовать свои имена. Атрибут с одинарным подчеркиванием _name был добавлен в класс Employee. Names default to None unless a string argument is passed during instantiation. Внутри класса Employee:

1. Определите метод getter под названием get_name(), который возвращает атрибут класса _name.
2. Добавьте метод setter. Внутри класса Employee:
 1. Определите метод setter set_name, который имеет дополнительный параметр для строки имени и устанавливает атрибут класса _name.
 2. Наконец, добавьте метод deleter. Внутри класса Employee:
 1. Определите метод deleter del_name, который удаляет атрибут.